

Особенности синтаксиса Java

Юрий Литвинов
y.litvinov@spbu.ru

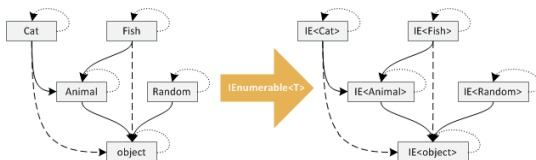
10.10.2026

Генерики в Java

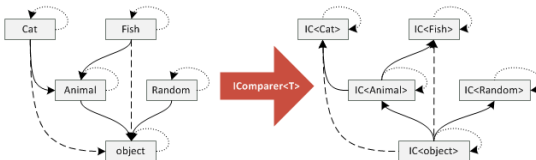
- ▶ Были добавлены только в Java 5, когда стало понятно, что хранить Object-ы и делать ручной каст — вообще не дело
 - ▶ Требовалось обеспечить обратную совместимость, поэтому формат байт-кода было решено не менять
 - ▶ Стирание типов
- ▶ Генерики в Java инвариантны, массивы ковариантны

Небольшое отступление про вариантность

Ковариантность:



Контравариантность:



<http://tomasp.net/blog/variance-explained.aspx/>

Почему это вообще важно

```
import org.apache.commons.lang3.tuple.MutablePair;
```

```
public void f(MutablePair<Object, Object> x)
{
    ...
}
```

```
f(new MutablePair<Object, Object>(apple1, apple2));
f(new MutablePair<Apple, Apple>(apple1, apple2)); // Ошибка компиляции
```

Чтобы нельзя было делать так:

```
public void f(MutablePair<Object, Object> x)
{
    x.SetLeft(new Battleship());
}
```

Массивы ковариантны?

```
String[] strings = new String[]{"ololo"};
```

// Всё ок, массив строк является массивом объектов, ковариантность

```
Object[] objectArray = strings;
```

```
objectArray[0] = 42; // ArrayStoreException в рантайме
```

Иначе пришлось бы делать так:

```
void printNames(Animal[] animals) { ... }
```

```
...
```

```
Dog[] dogs = new Dog[] {new Dog(), new Dog()};
```

```
Animal[] animals = new Animal[dogs.length];
```

```
for (int i = 0; i < dogs.length; i++) {
```

```
    animals[i] = dogs[i];
```

```
}
```

```
printNames(animals);
```

В С# то же самое, но это не «фича», а legacy языкового дизайна.

Wildcards

- ▶ ? **extends** T — какой-то подтип T
 - ▶ Можно читать как T, нельзя записывать ничего, кроме null
 - ▶ Ковариантность
- ▶ ? **super** T — надтип T (запись)
 - ▶ Можно записывать T и его подтипы, читать только как Object
 - ▶ Контравариантность
- ▶ Правило PECS: Producer Extends, Consumer Super
- ▶ Ограничения на параметры-типы: <T **extends** Comparable<T>>
— T обязан реализовать интерфейс
- ▶ В методах, обрабатывающих коллекции, почти всегда нужен wildcard, а не конкретный тип

Стирание типов

- ▶ Генерики существуют только на этапе компиляции
 - ▶ Компилятор проверяет и «стирает» параметры
 - ▶ Есть ограничение — параметр-тип заменяется на верхнюю границу, нет — на Object
- ▶ Поэтому нельзя:
 - ▶ Создать массив генерик-типа: `new T[]` не компилируется
 - ▶ Перегрузить метод, различающийся только параметром-типом
 - ▶ Вызвать `instanceof List<String>` — только `instanceof List<?>`
 - ▶ Создать статическое поле типа параметра-типа
 - ▶ Статическое поле одно на класс, а для всех инстанций генериков класс один
- ▶ Приведение к параметру-типу может привести к `ClassCastException` в рантайме
- ▶ Чем хорошо: не нужен каст при передаче в метод, принимающий сырую коллекцию (старый код)

Прочие особенности генериков

- ▶ Bridge-методы — обеспечивают корректный вызов переопределённых в потомке методов
- ▶ Обобщённые методы: `<T> T max(List<T> list)`
 - ▶ Параметр выводится из аргументов при вызове
- ▶ Типы-обёртки: **int** в генерике автоматически становится `Integer`
 - ▶ Строго говоря, генерики value-типов невозможны

Bridge-метод, пример

```
class Base<T> {  
    public void doWork(T t) { ... } // Стирается до Object  
}
```

```
class Derived extends Base<String> {  
    @Override // Вообще другой метод после стирания  
    public void doWork(String s) { ... }  
}
```

Bridge-метод (генерируется компилятором в Derived):

```
public void doWork(Object t) {  
    doWork((String) t);  
}
```

Enum как класс

- ▶ Перечисление — не список констант, а полноценный тип
- ▶ Компилятор создаёт класс, наследующий `java.lang.Enum<E>`
- ▶ Автоматически генерируются методы
 - ▶ `values()` — массив всех констант
 - ▶ `valueOf(String)` — константа по имени
- ▶ Константы — статические поля этого класса
 - ▶ Технически это значения самого класса (паттерн «Singleton»)
 - ▶ `enum` в Java ссылочные!
- ▶ В отличие от констант через **static final**:
 - ▶ Типобезопасность (нельзя передать `int` вместо цвета)
 - ▶ Возможность добавлять поля и методы
 - ▶ Можно использовать в `switch` и в генериках

Поля, методы и конструкторы в enum

- ▶ В enum можно объявлять поля и методы — обычные и статические
- ▶ Конструктор enum всегда private
 - ▶ Явный **private** — ошибка компиляции
 - ▶ Неявный — конструктор package-private, но вызвать всё равно можно только из тела enum
- ▶ Конструктор вызывается при инициализации констант, поэтому константы объявляются первыми
- ▶ Данные в константах — основной способ использования
 - ▶ Каждая константа несёт свои данные — технически она объект класса

Enum, пример

```
public enum Color {  
    RED(255, 0, 0),  
    GREEN(0, 255, 0),  
    BLUE(0, 0, 255);  
  
    private final int r, g, b;  
  
    Color(int r, int g, int b) {  
        this.r = r;  
        this.g = g;  
        this.b = b;  
    }  
  
    public String hex() {  
        return String.format("#%02x%02x%02x", r, g, b);  
    }  
}
```

У элементов enum тоже могут быть методы!

```
public enum Operation {  
    PLUS {  
        @Override  
        public double apply(double x, double y) { return x + y; }  
    },  
    MINUS {  
        @Override  
        public double apply(double x, double y) { return x - y; }  
    };  
  
    public abstract double apply(double x, double y);  
}
```

- ▶ Тут enum раскроется компилятором в иерархию классов
- ▶ Бесплатная реализация паттерна «Стратегия»

EnumSet и EnumMap

- ▶ `EnumSet<E>` — множество на базе битовой маски констант
- ▶ `EnumMap<K, V>` — отображение с массивом вместо хеш-таблицы
- ▶ Преимущества над `HashSet/HashMap`:
 - ▶ Быстрее: доступ по индексу вместо хеширования
 - ▶ Экономнее память: нет объектов-ключей и служебных структур
- ▶ Безопаснее: принимают только нужный `enum`, не любые объекты
- ▶ Почти всегда предпочтительнее, если ключ/элемент — `enum`
- ▶ `EnumSet.noneOf(Color.class)`,
`EnumSet.of(Color.RED, Color.GREEN)`

Особенности работы с enum

- ▶ Сравнение — через `==`, не `equals`
 - ▶ Константы — синглтоны, `==` безопасен
 - ▶ `equals` тоже работает, но не нужен
- ▶ Нельзя менять порядок объявления констант, если код использует `ordinal()`
 - ▶ И незачем его использовать — он для реализации `EnumSet/EnumMap` и прочей внутренней машинерии
- ▶ В `switch`-выражениях компилятор проверяет, что все варианты `enum`-а покрыты, в старых `switch` — нет

Почему enum-ы в Java такие

- ▶ Типобезопасность: enum-у нельзя присвоить произвольный int даже с кастом, это физически разные типы
- ▶ Enum — полноценное средство моделирования предметной области:
 - ▶ Связанные данные хранятся прямо в константах
 - ▶ Можно определять поведение, специфичное для enum, прямо в enum (инкапсуляция)
 - ▶ Можно определять поведение, специфичное для каждой константы, и использовать полиморфизм
 - ▶ В других языках enum часто используется как антипаттерн «Тэг типа»
- ▶ Получился побочный эффект, что enum даже с одним элементом полезен — бесплатная реализация паттерна «Одиночка» с безопасным многопоточным доступом
- ▶ Близкий аналог — размеченные объединения в функциональных языках

Вложенные классы

- ▶ Static nested class — объявлен static внутри внешнего класса
 - ▶ Не имеет доступа к **this** внешнего объекта
 - ▶ Ведёт себя как обычный вложенный класс в других языках
- ▶ Inner class — без static
 - ▶ Неявно хранит ссылку на внешний объект (Outer.this)
 - ▶ Для создания нужен экземпляр внешнего класса
 - ▶ Inner class суть часть целого (отношение композиции)
- ▶ Почему Inner class-ы так сделаны:
 - ▶ Для listener-ов внутри GUI-компонентов, чтобы они могли менять состояние компонента при наступлении события
 - ▶ Особенно это полезно для анонимных inner class-ов, которые долгое время играли роль лямбд в Java — OuterClass.this играл роль «замыкания для бедных»
 - ▶ Для поддержки итераторов — итератор имеет доступ к состоянию коллекции

Анонимные классы

- ▶ Безымянный класс, наследующий/реализующий тип
- ▶ Синтаксис: **new** Interface() { ... } или **new** Superclass(args) { ... }
- ▶ Можно объявлять собственные поля и методы, но они не видны снаружи
 - ▶ Раз имени у класса нет, кастать не к чему
- ▶ Экземпляр создаётся в точке объявления
- ▶ Использовались как простой аналог лямбд

Анонимные классы, пример

```
Runnable r = new Thread() {  
    @Override  
    public void run() {  
        System.out.println("Привет, мир!");  
    }  
};
```

```
Comparator<String> c = new Comparator<String>() {  
    @Override  
    public int compare(String s1, String s2) {  
        return s1.length() - s2.length();  
    }  
};
```

Захват переменных

- ▶ Анонимный класс может обращаться к переменным метода
- ▶ Но только если переменная `effectively final` — не изменяется после инициализации
 - ▶ Компилятор копирует значение в поле класса-замыкания
- ▶ Изменить переменную метода из анонимного класса нельзя, поле объемлющего класса — можно
 - ▶ Анонимный класс — честный Inner class, так что имеет `OuterClass.this`

Локальные классы

- ▶ Объявляются внутри метода, видны только в нём
- ▶ Сочетают свойства анонимных классов и именованных классов
 - ▶ Есть имя, можно создать несколько экземпляров
 - ▶ Реализуют интерфейс или расширяют класс
- ▶ Могут обращаться к переменным метода, но только если они `effectively final`

Доступ к private-членам

- ▶ Вложенные и локальные классы могут обращаться к private-полям и методам внешнего класса
 - ▶ Технически private-поля они читать не могут, компилятор генерирует Bridge-методы
 - ▶ На уровне байт-кода обращаться к private-членам нельзя, JVM не даст, поэтому некий костыль
- ▶ Концептуально вложенный класс — часть реализации внешнего
- ▶ Анонимные и локальные классы, объявленные в статических методах, OuterClass.this, естественно, не имеют

Подводные камни

- ▶ Потенциальное нарушение инкапсуляции, поведение размазывается по разным классам
- ▶ Вложенный класс держит ссылку на объемлющий, не давая сборщику мусора его собрать
 - ▶ Пока доступен хоть один итератор, список не удалится — потенциальная утечка памяти
 - ▶ Особенно опасно, если это listener, который подписался на какое-то событие из библиотеки, и держит свой this объемлющего класса до конца программы
- ▶ Неочевидные зависимости по данным между классами
 - ▶ У объемлющего класса нет возможности гарантировать, что в его состоянии не полезут внутренние классы
- ▶ Всё, что можно вынести на верхний уровень без потери смысла — лучше вынести: меньше связности и удобнее тестировать

Функциональные интерфейсы

- ▶ Функциональный интерфейс — интерфейс ровно с одним абстрактным методом
- ▶ `@FunctionalInterface` — проверка компилятором
 - ▶ Гарантирует, что интерфейс не «сломается» при добавлении метода
- ▶ Стандартные функциональные интерфейсы (`java.util.function`):
 - ▶ `Function<T, R>` — функция
 - ▶ `Predicate<T>` — предикат, возвращает `boolean`
 - ▶ `Supplier<T>` — поставщик, `Consumer<T>` — потребитель
 - ▶ `BiFunction`, `UnaryOperator` и др.
- ▶ Можно писать свои

Лямбды

- ▶ Анонимные функции для реализации функциональных интерфейсов
 - ▶ По сути, синтаксический сахар для анонимных классов
 - ▶ Но реализации несколько отличаются, лямбда может побыстрее работать
- ▶ Синтаксис: (параметры) -> тело
 - ▶ Например, `Runnable r = () -> System.out.println("Hi")` или `list.stream().filter(s -> s.length() > 3)`
- ▶ Замыкания — как в анонимных классах, effectively final-переменные и поля объемлющего класса
 - ▶ Лямбда имеет `OuterClass.this` и держит на него ссылку
 - ▶ В C# захват переменных «по ссылке», так что нет ограничения на effectively final, но есть способ прострелить себе ногу
- ▶ Лямбда — объект

Ссылки на методы

- ▶ Сокращение от лямбды, когда тело — просто вызов метода: `String::length`
- ▶ Четыре формы:
 - ▶ Статический метод: `Integer::parseInt`
 - ▶ Метод экземпляра привязанного объекта: `string::length` — метод запускается от объекта `string`
 - ▶ Метод экземпляра непривязанный: `String::length` — метод запускается от объекта, который ему передадут первым аргументом при вызове
 - ▶ Например, `list.stream().map(String::length)`
 - ▶ Конструктор: `ArrayList::new`
- ▶ Ссылки на методы могут быть несколько хуже по читаемости
 - ▶ Тем более что статические методы принимают аргументы как есть, а нестатические непривязанные принимают первым аргументом `this`

Ограничения лямбд

- ▶ Нельзя объявить поля и вспомогательные методы — тело может быть только выражением или блоком (с фигурными скобками)
- ▶ Нет собственного **this** — это **this** окружающего контекста
 - ▶ Ну им и не надо, раз у них полей и методов нет
- ▶ Нельзя пробрасывать checked-исключения

```
Runnable r = () -> { throw new IOException(); }; // не компилируется
```

 - ▶ Разумно, checked-исключения негде задекларировать
 - ▶ Обходной путь — обернуть в unchecked или свой функциональный интерфейс с throws в сигнатуре
- ▶ Нельзя создать переменную с именем, занятым параметром лямбды

Типизация лямбд

- ▶ Целевой тип выводится из контекста
 - ▶ Ищется функциональный интерфейс, метод которого принимает значения нужных типов и возвращает значение нужного типа
 - ▶ Если их несколько — ошибка компиляции
 - ▶ Тогда явно указываем тип: `f((FuncInt) () -> System.out.println("Hi"));` или `FuncInt lambda = () -> System.out.println("Hi");`
- ▶ Лямбда не создаёт класс в исходном коде
 - ▶ В байткоде — bootstrap-метод + вызов `invokedynamic`
 - ▶ При первом вызове JVM решает, что делать (сгенерить реальный объект, заинлайнить, ...)
 - ▶ При последующих вызовах переиспользуется результат первого
- ▶ Анонимный класс и лямбда ведут себя по-разному при захвате **this**
 - ▶ Если лямбда ничего не захватывает, `this` не хранится
 - ▶ Игра-угадка для сборки мусора
 - ▶ Не полагайтесь на оптимизацию захвата `this`, следите за памятью

Анонимный класс или лямбда

- ▶ Лямбда предпочтительнее для одного функционального метода: короче, компактнее
- ▶ Анонимный класс лучше, когда:
 - ▶ Нужно состояние — собственные поля, изменяемые между вызовами
 - ▶ Реализуется несколько методов (не функциональный интерфейс)
 - ▶ Нужна перегрузка конструктора суперкласса с разными аргументами
- ▶ Исторически: анонимные классы — до Java 8, лямбды — основной способ с Java 8

Иерархия исключений

- ▶ Throwable — всё, что можно бросить и поймать
 - ▶ Error — фатальные ошибки JVM и ресурсов (OutOfMemoryError, StackOverflowError), обычно не обрабатываем
 - ▶ Exception — предсказуемые, но не ожидаемые ошибки, обрабатываем
- ▶ RuntimeException и его наследники — unchecked
 - ▶ NullPointerException, IndexOutOfBoundsException, IllegalArgumentException
- ▶ Остальные Exception — checked
 - ▶ IOException, SQLException, ParseException
- ▶ Ошибка программиста — это unchecked, ошибка среды — чаще checked
- ▶ Если ошибка ожидаема и может возникнуть при нормальной работе программы (например, ошибка пользовательского ввода) — исключения не используем

Checked и unchecked исключения

- ▶ Checked: компилятор заставляет либо перехватить (**catch**), либо объявить **throws**
- ▶ Аргументы «за» checked:
 - ▶ Невозможно пропустить обработку ошибки, контракт виден в сигнатуре и проверяется компилятором
- ▶ Аргументы «против»:
 - ▶ Размывают контракт: исключение может всплыть из любой глубины
 - ▶ Побуждают к пустым catch-блокам и бросанию наружу через десятки методов
 - ▶ Неудобно в функциональном стиле и лямбдах
- ▶ Современный тренд (в т.ч. в новых API JDK) — отдавать предпочтение unchecked
- ▶ Checked — восстанавливаемые ошибки (перечитать файл, повторить запрос)

Работа с исключениями, хорошие практики

- ▶ Сохранять причину: **throw new** `IllegalStateException(e)`
 - ▶ Или `initCause/getCause`, **new** `Ex(e, message)` в собственных исключениях
- ▶ Не терять стек: не делайте **throw new** ... в `catch`
 - ▶ Тем более без `initCause`
- ▶ Писать собственные исключения, если стандартные слишком общие
- ▶ Не использовать исключения для управления потоком выполнения
- ▶ Исключение — не бесплатное, занимает время и память
 - ▶ Стек в Java может быть очень длинным

Expression switch

```
enum Level { DEBUG, INFO, WARN, ERROR }
```

```
int priority = switch (level) {  
    case DEBUG -> 1;  
    case INFO -> 2;  
    case WARN -> {  
        System.out.println("Warning level detected");  
        yield 3;  
    }  
    case ERROR -> {  
        if (critical) {  
            yield 5;  
        } else {  
            yield 4;  
        }  
    }  
};
```

Pattern matching и выражения

- ▶ Pattern matching для **instanceof** (Java 16):
 - ▶ **if** (obj **instanceof** String s) — переменная s уже типизирована
 - ▶ Похоже на smart cast в Kotlin, но требует нового имени
- ▶ Pattern matching для **switch** (Java 21):
 - ▶ **switch** (obj) { **case** String s -> ...; **case** Integer i -> ...; }
- ▶ Текстовые блоки (Java 15): `""" ... """`
 - ▶ Для SQL, HTML, JSON без `\n`

Record и sealed

- ▶ **record** (Java 16) — класс для хранения данных
 - ▶ Конструктор, геттеры (x(), не getX()), equals/hashCode/toString — автоматически
 - ▶ Поля финальные, класс финальный
 - ▶ Используется для хранения немутабельных данных (Data Transfer Objects)
 - ▶ Спасает от кучи boilerplate-кода
- ▶ **sealed** (Java 17) — контролируемое наследование
 - ▶ Перечисляет разрешённых наследников: permits Shape, Circle
 - ▶ В паре с pattern matching компилятор проверяет полноту switch
 - ▶ enum-ы как раз sealed
 - ▶ В Kotlin тоже есть sealed, но там все наследники должны быть в одном файле, в Java они явно перечисляются и могут быть где угодно

Implicitly declared classes

- ▶ JEP 445 и далее (Java 21+ preview, стабилизировано в Java 25)
- ▶ Тело main можно писать без объявления класса и метода:

// раньше

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello!");  
    }  
}
```

// теперь (Java 21+, preview)

```
System.out.println("Hello!");
```

- ▶ Компилятор неявно объявляет класс и main
- ▶ Пока дополняет, а не заменяет обычный стиль: библиотечный код, модули и привычная структура остаются прежними
- ▶ Языковой дизайн «Хочу как в Python» — минимизировать накладные расходы при написании простых программ
 - ▶ В C# так же

Рефлексия вообще

- ▶ Механизм получения compile-time-информации во время выполнения
- ▶ В разных языках имеет разные возможности (от очень ограниченной RTTI в C++ до весьма развитой в .NET)
 - ▶ В интерпретируемых языках (JavaScript, Python) не нужна — программа и так знает о себе всё
- ▶ Нужна много где, от создания плагиновых систем в приложениях до сериализации и юнит-тестов
- ▶ Реализуется хранением информации о типах в скомпилированной программе
 - ▶ В нашем случае в байт-коде
 - ▶ В Java рефлексия «уважает» стирание типов — параметры-типы генериков ей недоступны
- ▶ Как правило, язык дополнительно расширяется аннотациями (или атрибутами)
 - ▶ Произвольная информация, «надеваемая» на сущности программы, которую можно использовать в рантайме

Class и получение типов

- ▶ Объект `Class<T>` — описание типа в рантайме
- ▶ Три основных способа получить его:
 - ▶ `obj.getClass()` — у существующего объекта
 - ▶ `T.class` — по имени типа
 - ▶ `Class.forName("com.example.Foo")` — по строке, загружает класс
- ▶ Через `Class` открываются методы, поля, конструкторы, аннотации, generic-суперклассы
- ▶ Есть загрузчик класса (`ClassLoader`), именно он грузит классы из `CLASSPATH`, его можно использовать вручную

Доступ к полям и методам

- ▶ Получение членов класса:
 - ▶ `getDeclaredMethods()`, `getDeclaredFields()`
 - ▶ Также `getDeclaredConstructors()`; объявленные в классе без учёта наследования
 - ▶ Для наследуемых — `getMethods()`
- ▶ Приватные члены доступны, но надо вызвать `setAccessible(true)`, чтобы ими воспользоваться
- ▶ Создание объекта:
`clazz.getDeclaredConstructor().newInstance()`
 - ▶ Странное «clazz» используется как имя для переменной с классом, потому что **class** — ключевое слово
- ▶ Вызов метода по строке: `method.invoke(obj, args)`

Аннотации и стирание

- ▶ Аннотации доступны рефлексией, только если retention — RUNTIME
 - ▶ SOURCE — виден компилятору (Override, SuppressWarnings)
 - ▶ Линтеры, кодогенераторы и т.п.
 - ▶ CLASS — остаётся в class-файле, но не виден в рантайме
 - ▶ Для инструментов, работающих с байт-кодом (AOT-компиляторы, обфускаторы и т.п.)
 - ▶ Retention — аннотация к аннотациям, указывается при объявлении аннотации
- ▶ Чтение: `clazz.getAnnotation(Deprecated.class)`, аннотации на методах и параметрах
- ▶ Стирание проявляется и здесь:
 - ▶ Нельзя получить тип параметра поля: `List<String> field` виден рефлексией только как `List`
- ▶ Аннотации не влияют на поведение сами по себе

Как использовать

Объявление аннотации:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyTest { }
```

Использование аннотации:

```
public class TestRunner {
    public static void runTests(Object instance) throws Exception {
        for (Method m : instance.getClass().getMethods()) {
            if (m.isAnnotationPresent(MyTest.class)) {
                m.invoke(instance);
            }
        }
    }
}
```

Ограничения и цена рефлексии

- ▶ Производительность: вызов через рефлексия заметно медленнее прямого
 - ▶ MethodHandles (с Java 7) быстрее, но не так гибко
- ▶ Инкапсуляция и модули JPMS:
 - ▶ С Java 9 доступ к internals других модулей ограничен
 - ▶ `setAccessible(true)` может выбросить `InaccessibleObjectException`
- ▶ Слабая типобезопасность: ошибки имен классов и методов ловятся в рантайме
- ▶ Хрупкость: переименование поля ломает все обращения по строкам
- ▶ Безопасность: рефлексия позволяет обходить приватность, поэтому её ограничивают в песочницах и в `module-info.java`

Lombok

- ▶ Библиотека, генерирующая код на этапе компиляции
- ▶ Основные аннотации:
 - ▶ **@Data** — геттеры, сеттеры, equals/hashCode/toString, конструктор из полей
 - ▶ **@Getter** / **@Setter** — точноно
 - ▶ **@Value** — только геттеры, equals/hashCode/toString, конструктор из полей
 - ▶ В современной Java есть **record**
 - ▶ **@Builder** — сгенерируется реализация паттерна «Builder»
 - ▶ **@NoArgsConstructor** / **@AllArgsConstructor**
- ▶ Спасает от кучи boilerplate-кода, особенно из-за отсутствия property в Java

Как работает Lombok

- ▶ Annotation Processing (JSR 269): компилятор вызывает обработчик аннотаций и вставляет сгенерированный код в AST
 - ▶ По сути, это плагин к компилятору, реализующий `javax.annotation.processing.Processor`
 - ▶ Можно (и легко) свой написать
- ▶ На выходе обычный class-файл с дополнительными методами
 - ▶ Ничего не происходит в рантайме
- ▶ Отсюда особенности:
 - ▶ Нужна поддержка IDE (плагин Lombok) — иначе не видны сгенерированные методы
 - ▶ Сложнее отлаживать: в исходниках метода нет
 - ▶ Lombok «не знает» о ваших методах, написанных руками, и может конфликтовать
 - ▶ Так что record предпочтительнее